

Introducción

El código fuente llamado Firmware realizado para la EGA se divide en 10 tareas:

task_init: Esta tarea inicializa todos los periféricos necesarios para el proyecto, luego se elimina.

task_Setpoint: Esta tarea configura el setpoint del sistema y se comunica con varias tareas.

task_hcsr04: Esta tarea mide la altura y envía el resultado a la tarea ***task_leds***

task_guardiana_sd: Esta tarea almacena la configuración y los eventos de alerta

task_guardiana_lcd: Esta tarea se encarga de mostrar en pantalla la configuración y además indicar que estamos configurando en cada momento.

task_debounce_boton: Esta tarea es de sensar el botón

task_guardiana_leds: Son leds indicadores que alertan de que los límites se han superado, si esto ocurre se envía por cola el dato para ser almacenado en la SD.

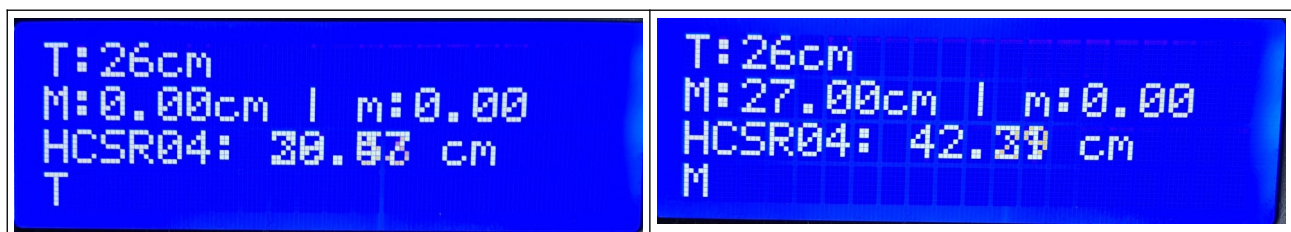
task_rtc: Esta tarea solo captura el momento en que se genera un evento

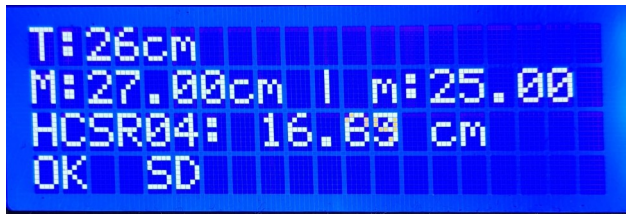
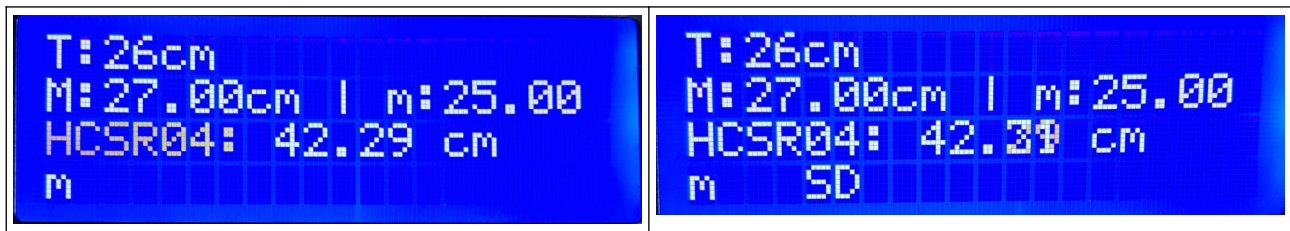
task_pid: Es la tarea de control del sistema.

Carga y visualización de datos

La carga de datos se realiza por medio del potenciómetro ***J11*** (según esquemático) mientras que el botón ***SW1*** realiza el cambio de opción. En cada opción, el LCD, indicará donde estamos posicionados, por ejemplo si estoy en la opción de configurar TARGET el LCD mostrará en la línea 4, columna 0 el carácter T. El siguiente cuadro muestra que significa cada carácter visualizado en la línea 4.

Carácter	Indicación	Posición en LCD
T	Configurando TARGET	L=4, C=0
M	Configurando límite máximo	L=4, C=0
m	Configurando límite mínimo	L=4, C=0
OK	Configuración confirmada	L=4, C=0
SD	Configuración guardada en SD	L=4, C=3





Las opciones se agrupan en un algoritmo *switch-case* siendo en total 5 opciones (case1, case 2, case3, case4, case0). Al recorrer cada opción enviamos información al LCD indicando el parámetro a configurar mientras que la línea 4 nos informa que parámetro estamos configurando. Esta información se concentro en un único tipo de dato del tipo estructura, la estructura creada se puede ver en la Figura 1.

```
typedef struct
{
    uint32_t setpoint;
    float setpoint_min;
    float setpoint_max;
    float altura_medida;
    uint8_t linea;
    uint8_t guardado;
}estructura_setpoint;
```

Figura 1. Tipo de dato

Los campos son los siguientes:

setpoint: Este campo indicara el TARGET, es decir que altura estará la esfera.

setpoint_min: Este campo indicara el mínimo de altura, sera usado para la alerta mínima.

setpoint_max: Este campo indicara el máximo de altura, sera usado para la alerta máxima.

linea: Este campo se utiliza para indicar en la *task_guradiana_lcd* que opción estoy configurando, solo afecta a la línea 4 del LCD

guardado: Este campo nos permite indicar si lo que vamos a guardar es la configuración (0), el alerta de mínimo (2), o el alerta de máximo (1).

Opción 1

Esta opción nos permitirá indicar el TARGET mediante un potenciómetro que utiliza el ADC para convertir tensión a un valor entero, luego ese valor entero sera usado para convertirlo en una altura. En el inicio esta opción suspenderá la tarea *task_guardiana_leds* de manera que esta no consuma recursos hasta tener los limites fijados.

```

case 1:
    //Configuro el setpoint
    xQueueReceive(queue_sd, &aux, pdMS_TO_TICKS(0)); //Borra la cola de task_guradina_sd
    xQueueReceive(queue_leds, &aux, pdMS_TO_TICKS(0)); //Borra la cola de task_guradiana_leds
    vTaskSuspend(taskLEDS); //Suspendo a task_guradina_leds hasta finalizar la configuracion
    data.guardado = 0;
    valor_adc = adc_read();
    tension = (valor_adc * 3.3f) / 4095;
    valor_altura = ((valor_adc * 3.3f) / 4095)*10;
    data.setpoint = valor_altura;
    data.linea = 1;
    if(valor_altura > 28)
    {
        data.setpoint = 28;
    }
    if(valor_altura == 0)
    {
        data.setpoint = 5;
    }
    break;

```

Figura 2. Contenido de la opción 1

Como muestra la Figura 2, el campo **linea** tiene asignado el valor 1, esto indicara al LCD que imprima en la linea 4 el contenido T. El contenido del **task_guardiana_lcd** se puede ver en la Figura 3. en este caso como el campo tiene el valor 1, cuando la tarea actual envié el dato al LCD por medio de la cola **queue_setpoint**, la tarea del LCD tomara el dato de la cola y asignara el contenido del campo **linea** a una variable local llamada **linea4**. La variable **linea4** se usara en el **switch-case** de la tarea del LCD, indicando que estamos tratando de indicar el TARGET.

Este procedimiento se repetirá en todas las opciones de la tarea **task_Setpoint**, de manera que en todo momento sabremos que estamos configurando.

```

while (true)
{
    if(xSemaphoreTake(sem_mutexi2c,portMAX_DELAY) == pdTRUE)
    {
        xQueueReceive(queue_setpoint, &repcion_lcd, pdMS_TO_TICKS(100));
        xQueueReceive(queue_hcsr04, &val_hcsr04, pdMS_TO_TICKS(100));
        linea4 = repcion_lcd.linea;
        lcd_set_cursor(0, 0);
        sprintf(buffer, "T:%lucm          ", repcion_lcd.setpoint);
        lcd_string(buffer);
        lcd_set_cursor(1, 0);
        sprintf(buffer, "M:%.2fcm | m:%.2f", repcion_lcd.setpoint_max, repcion_lcd.setpoint_min);
        lcd_string(buffer);
        lcd_set_cursor(2, 0);
        sprintf(buffer, "HCSR04: %.2f cm    ", val_hcsr04);
        lcd_string(buffer);
        switch (linea4)
        {
            case 1:
                lcd_set_cursor(3,0);
                lcd_string("T ");
                lcd_set_cursor(3,4);
                lcd_string(" ");
                break;
            case 2:
                lcd_set_cursor(3,0);
                lcd_string("M ");
                break;
            case 3:
                lcd_set_cursor(3,0);
                lcd_string("m ");
                break;
            case 4:
                lcd_set_cursor(3,4);
                lcd_string("SD");
                break;
            case 0:
                lcd_set_cursor(3,0);
                lcd_string("OK ");
                break;
            default:
                break;
        }
    }
    xSemaphoreGive(sem_mutexi2c);
    vTaskDelay(pdMS_TO_TICKS(50));
}

```

Figura 3. Parte del contenido de task_guardiana_lcd

Opción 2 y Opción 3

Estas dos opciones permite fijar los limites máximo y mínimo, su funcionamiento es idéntico, la única diferencia es que el campo linea sera diferente, siendo 3 para el mínimo y 2 para el máximo. En estas opciones se agregaron limites por defecto, siendo que los limites se ajustaran en setpoint+1 para el máximo y setpoint-1 para el mínimo, esto no significa que no podamos variar los limites. Pero ademas nos limita a que en cada limite no exitan incongruencias, es decir que mi lmite máximo no sea menor al setpoint y mi limite mínimo no se mayor al setpoint.

```

case 2:
    valor_adc = adc_read();
    tension = (valor_adc * 3.3f) / 4095;
    valor_altura = ((valor_adc * 3.3f) / 4095)*10;
    data.setpoint_max = valor_altura;
    data.linea = 2;
    if(valor_altura > data.setpoint)
    {
        data.setpoint_max = valor_altura;
    }
    else
    {
        data.setpoint_max = data.setpoint + 1;
    }
    break;

```

```

case 3:
    valor_adc = adc_read();
    tension = (valor_adc * 3.3f) / 4095;
    valor_altura = ((valor_adc * 3.3f) / 4095)*10;
    data.linea = 3;
    if(valor_altura < data.setpoint)
    {
        data.setpoint_min = valor_altura;
    }
    else
    {
        data.setpoint_min = data.setpoint - 1.0f;
    }
    break;

```

Figura 4. Opciones 2 y 3

Opción 4

Esta opción toma la configuración y la guarda en el SD, en esta opción se usa el campo *guardado* cuyo valor será 0, indicando que nuestra intención es guardar la configuración. Si la cola pudo enviar entonces cede el semáforo binario para desbloquear la tarea *task_guardiana_sd*. Pero además se modifica el campo *linea* para indicar por el LCD que se realizó el guardado de la configuración.

```

case 4:
    //Guardo configuracion en la SD
    data.linea=4;
    data.guardado = 0; //Indico que guardo setpoint en la SD
    if (xQueueSend(queue_sd, &data, pdMS_TO_TICKS(0)) == pdPASS)
    {
        xSemaphoreGive(sem_memoriaSD);
    }
    else
    {
        //printf("Setpoint GUARDADO\n");
        data.guardado = 2;
    }
    break;

```

Figura 5. Contenido de la Opción 4

Opción 0

Esta última opción enviara por cola la configuración a la tarea *task_pid*, pero también enviara por la los límites a la tarea *task_guardiana_leds*, si la cola *queue_leds* envía con éxito entonces la tarea de leds deja de estar suspendida y pasa al estado ready lista para sensor los estado de alerta y guardarlos en la SD.

En todo momento se está enviando al LCD información por medio de la cola *queue_setpoint* pero al finalizar la selección, la opción quedara en 0. Pero si el botón se vuelve a pulsar la copia para a 1, limpiando colas y suspendiendo la tarea de leds. Comenzando todo el proceso de nuevo.

```

case 0:

    if( xQueueSend( queue_leds, &data, pdMS_TO_TICKS(0)) == pdPASS )
    {
        vTaskResume( taskLEDS);
    }
    else
    {
        //printf("Sensado de alturas limites activado\n");
    }
    data.linea = 0;
    xQueueSend( queue_pid, &data, pdMS_TO_TICKS(0));
    xQueueReceive( queue_sd, &aux, pdMS_TO_TICKS(0));
    break;

```

Figura 6. Contenido de la Opción 0

Uso de las alertas

Para esta parte usamos la tarea *task_guardiana_leds* la cual indicara por medio de los leds si los limites son superados. Esta tarea envía por la cola *queue_superada* la información del limite que se supero y en cuanto se supero, para ser almacenado en la memoria SD. Recibe información de dos tareas; *task_pid*, *task_hcsr04*. Cuando uno de los limites es superado entonces se desbloquea la tarea *task_guardiana_sd* la cual almacenara la información y se volverá a bloquear a la espera de un nuevo evento.

```

while(true)
{
    if( xQueuePeek( queue_leds, &data, portMAX_DELAY) == pdPASS )
    {
        //Alerta Maxima, cuando se supera el maximo permitido
        if(altura > data.setpoint_max)
        {
            gpio_put(GPIO_LED_MAX, true);
            data.altura_medida = altura;
            data.guardado = 1;
            if( xQueueSend( queue_superada, &data, pdMS_TO_TICKS(0)) == pdPASS )
            {
                xSemaphoreGive( sem_memoriaSD );
            }
        }
        if(altura < data.setpoint_max)
        {
            gpio_put(GPIO_LED_MAX, false);
            xQueueReceive( queue_superada, &aux, pdMS_TO_TICKS(0));
        }
        //Alerta Minima, cuando se essta por debajo del minimo permitido
        if(altura < data.setpoint_min)
        {
            gpio_put(GPIO_LED_MIN, true);
            data.altura_medida = altura;
            data.guardado = 2;
            if( xQueueSend( queue_superada, &data, pdMS_TO_TICKS(0)) == pdPASS )
            {
                xSemaphoreGive( sem_memoriaSD );
            }
        }
        if(altura > data.setpoint_min)
        {
            gpio_put(GPIO_LED_MIN, false);
        }
    }
}

```

Figura 7. Contenido del while para la tarea *task_guardiana_leds*

La cola *queue_superada* maneja un tipo de dato *estructura_setpoint*, mientras que la cola *queue_hcsr04* maneja un tipo de dato *float*.

Almacenamiento de datos

La tarea *task_guradiana_sd* sera la encargada de almacenar los datos en la memoria SD. Esta tarea es de prioridad 3 y solo realiza su acción en determinados evento; Guardar la configuración, Guardar el alerta máximo, Guardar alerta mínimo. Estos eventos están sincronizados por colas y por un semáforo binario. El semáforo binario sera cedido solo si hay lugar en la cola, una vez que es cedido la tarea *task_guradiana_sd* tomara el semáforo, realizara su acción de guardado y luego intentara tomar nuevamente el semáforo y se bloqueara, ya que lo tomo la principio.

Cada evento que activa la *task_guradiana_sd* esta identificado en el campo *guardado*, siendo 3 opciones; guardar configuración (0), guardar alerta máxima (1), guardar alerta mínima (2).

```
void task_guardiana_sd(void *params)
{ estructura_setpoint datasd, AlturaSuperada, borrado={.guardado=2};
  char buffer1[500], buffer2[100], buffer3[100];
  ds3231_time_t toma_fecha;
  uint8_t guardado;

  while(true)
  {
    xSemaphoreTake(sem_memoriaSD, portMAX_DELAY);
    printf("Dentro de task_guardiana_sd\n");
    xQueueReceive(queue_rtc, &toma_fecha, 0);
    xQueuePeek(queue_sd, &datasd, pdMS_TO_TICKS(0));

    xQueuePeek(queue_superada, &AlturaSuperada, pdMS_TO_TICKS(0));
    sprintf(buffer2, "%f", AlturaSuperada);
    sprintf(buffer3, "%d", datasd.guardado);

    if(datasd.guardado == 0)
    {
      guardar_sd(buffer1);
      printf("Setpoint guardado\n");
      datasd.guardado=2;
    }
    if(AlturaSuperada.guardado == 1)
    {
      guardar_sd(buffer2);
      printf("Maximo sueprado guardado\n");
    }
    if(AlturaSuperada.guardado == 2)
    {
      guardar_sd(buffer3);
      printf("Minimo no alcanzado guardado\n");
    }
  }
}
```

Figura 7. Contenido de la tarea *task_guardiana_sd*

Control PID

La tarea *task_pid* es la realiza el control de la altura con un tiempo de muestro de 30 mseg definido por la variable *DT*.

Variables e inicializacion

Antes de entrar al *while* hay una serie variables a inicializar.

TARGET: es el tipo float y se usara para almacenar el setpoint extraído del dato paortado por la cola queue_pid

last_pwm: almacena el ultimo valor de PWM aplicado, luego se utiliza este valor para implantar un filtro de histeresis.

fan: configura el uso para PWM

sensor: inicializa el sensor HC-SR04 para la medición de la altura.

pid: inicializa el PID con lo valores definidos de Kp, Ki y Kd, limMAX, limMin, ademas también se inicializa la variables integrador y diferenciador.

filtered_height: variable que almacena la distancia filtrada del sensor.

alpha: Constante del filtro para filtrar la medición del sensor en caso de ruido

Pasos para el control

Podemos separa el control PID en los siguientes pasos:

1. Se carga el campo setpoint a la variable TARGET
2. Se ejecuta la medición, pero se realiza de manera robusta a fin de mejorar el control y precisión del dato medido por el sensor. Se adapto la medición para que se hiciera considerando la medida desde el piso al centro de la pelota. Obtenida la medición se realiza el filtro para eliminar cualquier ruido.
3. Con la medición filtrada llamamos a la función ***PIDController_Update(&pid, TARGET, filtered_height, dt)***, pasándole los argumentos señalados. Esta función esta dentro de la librería del PID. Devolverá un valor del tipo float, pero como necesitamos usar PWM lo casteamos a un entero del tipo ***uint16_t***.
4. Limites del PWM: acá verificamos que el PWM obtenido este dentro de un limites de histerisis, si es así se realiza un promedio ponderado entre el actual PWM y el ultimo PWM (***last_pwm***) y luego se aplica dicho promedio al seteo del PWM, donde finalmente el PWM actual se guarda en el ***last_pwm***. Los limites del PWM se evalúan en todo momento si por alguna razón son superados se fijan en los valores máximo (***PWM_MAX***) o mínimo (***PWM_MIN***) según corresponda.

El controlador PID, en especial la Kd, es muy sensible al ruido por lo que una variación en la muestra de la medición puede causar que la salida cambie de drásticamente, acelerando y desacelerando el ventilador bruscamente.

La lógica que se usa para el suavizado no es una histeresis tradicional que usa umbrales, sino una filtro simple de cambio brusco basado en el ultimo valor conocido.

A. Detección del Salto

- ***if(abs(pwm - last_pwm > 100))***: Esta condición comprueba si el valor de PWM calculado por el PID (***pwm***) difiere del valor aplicado en el ciclo anterior (***last_pwm***) en más de 100 unidades (ya que el PWM máximo es 4999).
 - Si la diferencia es pequeña (menor o igual a 100), se acepta el nuevo valor del PID sin modificar.

- Si la diferencia es grande (mayor a 100), se considera un cambio brusco o un salto, y se aplica el suavizado.

B. Aplicación del Promedio Ponderado

- $pwm = (pwm + last_pwm * 2) / 3$; Si se detecta un salto, el nuevo valor de pwm no se aplica directamente. En su lugar, se calcula un promedio ponderado entre el nuevo valor (pwm) y el valor anterior ($last_pwm$).
 - Se le da el doble de peso al valor anterior ($last_pwm * 2$).
 - El nuevo valor de pwm se ve obligado a acercarse gradualmente al valor del PID, en lugar de saltar de golpe. Esto resulta en una transición más suave de la velocidad del ventilador.
- $PWM_{suavizado} = \frac{PWM_{nuevo} + (2 \times PWM_{anterior})}{3}$

3. Límites Finales

- $pwm = (pwm < MIN_PWM) ? MIN_PWM : (pwm > MAX_PWM) ? MAX_PWM : pwm$; Después del posible suavizado, se verifica que el valor final de pwm esté dentro de los límites de seguridad definidos ($MIN_PWM = 1800$ y $MAX_PWM = 4000$).

Este mecanismo logra el efecto de histéricos de la señal al oponerse activamente a los cambios rápidos (saltos), lo que hace que la respuesta del ventilador sea más estable y predecible.

Filtro pasa-bajos

Este filtro actúa a la entrada, es decir contiene los cambios bruscos en la medición del sensor que impactan en la respuesta PWM. El sensor está expuesto a ruido, por lo que se debe filtrar la información. Este filtro trabaja junto al conjunto de suavizado del PWM.

El filtro se aplica directamente a la medición de la altura ($current_height$) antes de que pase al controlador PID. El filtro utilizado es uno de los más simples siendo el filtro de primer orden o filtro exponencial móvil:

$$Altura_Filtrada = \alpha \times Altura_Actual_k + (1 - \alpha) \times Altura_Filtrada_{k-1}$$

Donde:

$Altura_Filtrada_k$: Es la nueva lectura suavizada que se usa en el PID

$Altura_Actual_k$: Mención cruda del sensor ($current_height$)

$Altura_Filtrada_{k-1}$: Es el valor suavizado del ciclo anterior.

α : ES el factor de suavizado, con un valor de 0.3

Como contrapartida por este conjunto de filtro más suavizado se tiene una respuesta más lenta. En este punto se tiene un compromiso velocidad-robustez

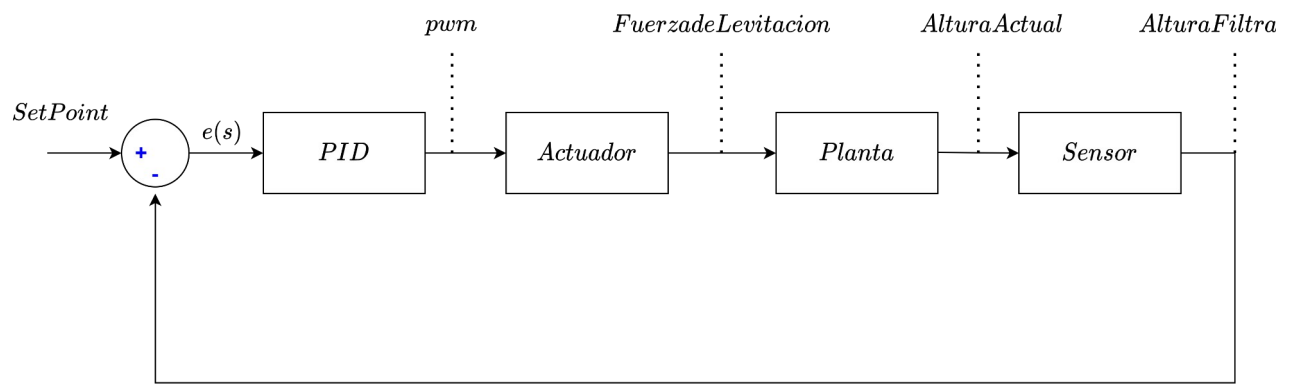


Figura 8.Diagrama en bloques del sistema de control